稀疏基整理

傅里叶相关变换基

离散傅里叶变换 DFT

DFT 定义

$$X(k)=\sum_{n=0}^{N-1}x(n)e^{-j\frac{2\pi}{N}kn}=\sum_{n=0}^{N-1}x(n)W_N^{kn}$$

矩阵表示:

$$\begin{bmatrix} X(0) \\ X(1) \\ X(2) \\ \vdots \\ X(N-1) \end{bmatrix}=\begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & W_N^1 & W_N^2 & \cdots & W_N^{N-1} \\ 1 & W_N^2 & W_N^4 & \cdots & W_N^{2(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & W_N^{N-1} & W_N^{2(N-1)} & \cdots & W_N^{(N-1)(N-1)} \end{bmatrix}\begin{bmatrix} x(0) \\ x(1) \\ x(2) \\ \vdots \\ x(N-1) \end{bmatrix}$$

归一化得基:

$$P=\frac{1}{\sqrt{N}}\begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & W_N^1 & W_N^2 & \cdots & W_N^{N-1} \\ 1 & W_N^2 & W_N^4 & \cdots & W_N^{2(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & W_N^{N-1} & W_N^{2(N-1)} & \cdots & W_N^{(N-1)(N-1)} \end{bmatrix}$$

Matlab 实现:

```
dftmtx(N)/sqrt(N)
```

快速傅里叶变换 FFT

使用库利-图基等算法简化 DFT 计算

Matlab 实现:

```
fft(eye(N))/sqrt(N)
```

离散余弦变换 DCT

类似于 DFT，但仅使用实数

DCT 定义

$$f_m = \sqrt{\frac{2}{n}} \sum_{k=0}^{n-1} x_k \cos\left[\frac{\pi}{n} m \left(k + \frac{1}{2}\right)\right]$$

Matlab 实现：

```
dctmtx(N)
```

离散沃尔什-哈达玛变换 DHT

DHT 与尺度为 2 的 DFT 事实等价

其变换基可由下式递归推导：

$$H_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

$$H_m = \frac{1}{\sqrt{2}} \begin{pmatrix} H_{m-1} & H_{m-1} \\ H_{m-1} & -H_{m-1} \end{pmatrix}$$

小波相关变换基

离散小波变换 DWT

使用不同的小波的平移和变换来匹配波形

常用小波有 Haar 小波、多贝西小波等

Matlab 实现：

```
dwt(x,wname)
% wname为使用的小波种类
```

啁啾变换 Chirplet transform

与波不同，啁啾为频率随时间变化的信号

[Matlab 实现]

> Steve Mann, Simon Haykin, [The Chirplet Transform: Physical Considerations], IEEE TRANSACTIONS ON SIGNAL PROCESSING,VOL. 43, NO. 11, NOVEMBER 1995

曲波变换 Curvelet transform

与小波不同，曲线波具有方向性，多分辨率等特性。

[Matlab 实现 1 介绍]

[Matlab 实现 2]

除上述方法外还有诸如 Bandlet， Contourlet 等变换

其它方法

学习字典 Learning Dictionary

给定字典$D=[d_1,d_2,...,d_K]\in\mathbb{R}^{n\times K}$，使得信号$X=[x_1,x_2,...,x_n]\in\mathbb{R}^{n\times N}$可被表示为

$$X\approx DA$$

其中$A=[\alpha_1,\alpha_2,...,\alpha_N]\in\mathbb{R}^{K\times N}$ 为X在D下的表示系数，当α_i中非零元素远小于K时，则称该向量为稀疏的。

则字典学习优化问题可表示为：

$$\min_{D,A}\{\|X-DA\|_F^2+\lambda_A g_A(A)\}$$

其中$g_A()$为求稀疏度函数，λ_A为归一化参数。

常见字典学习算法：

算法	主要思想
Sparsenet	最大似然估计
MOD	l_o范数衡量稀疏性 最小均方误差
K-SVD	正交匹配追踪(OMP) 奇异值分解(SVD)
ODL	每次迭代仅处理一个样本

> 练秋生, 石保顺, 陈书贞. 字典学习模型、算法及其应用研究进展. 自动化学报, 2015, 41(2): 240−260

重构算法整理

贪婪算法

追求局部最优，计算速度较快。

匹配追踪算法 MP

算法 1. MP 算法

1) 初始化冗余向量 $\boldsymbol{r}_0 = \boldsymbol{y}$, 迭代计数 $t = 1$;

2) 找到索引 λ_t, 使得:

$$\lambda_t = \arg \max_{j=1,2,\cdots,N} |\langle \boldsymbol{r}_{t-1}, \Phi_j \rangle|;$$

3) 计算新的近似 $\boldsymbol{a}_t$ 和冗余 $\boldsymbol{r}_t$:

$$\boldsymbol{a}_t = \langle \boldsymbol{r}_{t-1}, \Phi_{\lambda_t} \rangle \Phi_{\lambda_t}, \quad \boldsymbol{r}_t = \boldsymbol{r}_t - \boldsymbol{a}_t$$

4) $t = t + 1$, 如果 $t < K$, 返回第 2) 步.

从字典矩阵 D（也称为过完备原子库中），选择一个与信号 y 最匹配的原子(也就是某列)，构建一个稀疏逼近，并求出信号残差，然后继续选择与信号残差最匹配的原子，反复迭代，信号 y 可以由这些原子的线性和，再加上最后的残差值来表示。很显然，如果残差值在可以忽略的范围内，则信号 y 就是这些原子的线性组合。

正交匹配追踪算法 OMP

算法 2. OMP 算法

1) 初始化冗余向量 $\boldsymbol{r}_0 = \boldsymbol{y}$, 索引集合 $\Lambda_0 = \phi$, 迭代计数 $t = 1$;

2) 找到索引 λ_t, 使得:

$$\lambda_t = \arg \max_{j=1,2,\cdots,N} |\langle \boldsymbol{r}_{t-1}, \Phi_j \rangle|;$$

3) 令 $\Lambda_t = \Lambda_{t-1} \cup \{\lambda_t\}$;

4) 计算 $\{\Phi_\lambda : \lambda \in \Lambda_t\}$ 张成空间的正交投影 P_t;

5) 计算新的近似 $\boldsymbol{a}$ 和冗余 $\boldsymbol{r}$: $\boldsymbol{a}_t = P_t \boldsymbol{y}$, $\boldsymbol{r}_t = \boldsymbol{y} - \boldsymbol{a}_t$;

6) $t = t + 1$, 如果 $t < m$, 返回第 2) 步;

7) 获得的估计 $\widehat{s}_\lambda$ 在索引 Λ_m 位置的元非零, 且在该位置的测量向量逼近为: $\boldsymbol{a}_m = \sum_{\lambda \in \Lambda_m} \widehat{s}_\lambda \Phi_\lambda$.

OMP 算法是在 MP 算法的基础上进行改进的，其挑选原子的标准和 MP 算法一致，也就是在训练字典 A 里挑选和测试样本 x 最为匹配的字典原子。不相同之处在于：OMP 算法在每一次迭代过程

中对所挑选的全部原子先要执行 Schmidt 正交化操作，来确保每一次循环结果都是最优解。使得在同等精度的条件下，OMP 算法的性能更好，其收敛速度也更快。

OMP 每次迭代均只选择与残差最相关的一列。

> [压缩感知——浅析匹配追踪算法（Matching Pursuit，MP）与正交匹配追踪算法（Orthogonal Matching Pursuit，OMP）]

MP 算法与 OMP 算法具有相同的最优原子选择策略, 每次迭代均以测量矩阵 ϕ 中与当次冗余向量 r_{t-1} 具有最大内积的列向量作为待估信号的支撑候选. 但是两者在残差信号的更新方式上不同, MP 算法由其最新误差r_t 与当次所选原子 $\phi\lambda_t$的正交性($\langle r_t, \phi\lambda_t\rangle = \langle r_{t-1} - a_t, \phi\lambda_t\rangle \geq 0$) 来保证算法的收敛性, 然而这点仅保证了冗余向量r_t与 $\phi\lambda_t$的正交性,而不能保证r_t 与已选原子集合 $\phi\lambda_1, \phi\lambda_1, \ldots, \phi\lambda_1$的正交性, 这使得 MP 求得的近似解在最优 K 项近似的意义下都是次最优的, 这也是 MP 算法需要较多次数的迭代才能保证收敛的原因. OMP 算法通过递归的对已选原子集合正交化(见算法 2 中的第 4 步) 求出正交投影P_t, 利用$r_t = y_¡ P_t y$ 的残差更新方式,克服了 MP 算法的次最优性.

OMP 与 MP 的共同缺点在于原子选择机制相对于新的冗余误差的非最优性, 也就是说待选原子一旦进入支撑候选中, 则永久添加不会再被删除, 缺少“回溯”的思想. 这里“回溯”表示在当前迭代的步骤中, 仍对上次迭代中所入选的原子进行同步检验, 如果仍然满足当次最优意义则保留, 否则剔除作为下次待选. 其意义在于它可以最大程度保证重建的全局最优性, 因为在某次迭代中达到贪婪条件的原子并不能保证在以后迭代步骤中仍能达到, 所有原子都应做到“删除”和“添加”自由, 这种思想在后续产生的多种贪婪算法中得到了广泛的应用.

后退型最优正交匹配 IBOOMP.

算法 3. IBOOMP 算法

1) 初始化冗余 $\boldsymbol{r}_0 = \boldsymbol{y}$, 记 $\boldsymbol{\gamma}_j = \Phi_j, j = 1, 2, \cdots, N$, 取索引 $\lambda_1 = \arg\max_{1 \le j \le N} |\langle \Phi_j, \boldsymbol{y} \rangle|$, 索引集合 $\Lambda_1 = \{\lambda_1\}$, 正交投影函数 $\boldsymbol{\psi}_1 = \Phi_{\lambda_1} = \boldsymbol{\beta}_1$ 和分解系数 $c_1 = \langle \Phi_{\lambda_1}, \boldsymbol{y} \rangle$, 迭代计数 $t = 1$;

2) 对 $j = 1, 2, \cdots, N$, 分别计算:

$$\boldsymbol{\gamma}_j = \boldsymbol{\gamma}_j - \boldsymbol{\psi}_j \langle \boldsymbol{\psi}_t, \Phi_t \rangle, \ b_j = \langle \boldsymbol{\gamma}_j, \boldsymbol{y} \rangle,$$
$$d_j = d_j - |\langle \boldsymbol{\psi}_t, \Phi_j \rangle|^2, \ e_j = |b_j|^2 / d_j;$$

3) 令 $t = t + 1$, 寻找 λ_t, 使得 $\lambda_t = \arg\max_{1 \le j \le N} e_j$, 记索引集合 $\Lambda_t = \Lambda_{t-1} \cup \{\lambda_t\}$. 分别记:

$$\|\boldsymbol{R}_t\|^2 = \|\boldsymbol{R}_{t-1}\|^2 - e_{\lambda_t}, \boldsymbol{\psi}_t = \frac{\boldsymbol{\gamma}_{\lambda_t}}{\sqrt{d_{\lambda_t}}}, \boldsymbol{\beta}_t = \frac{\boldsymbol{\gamma}_{\lambda_t}}{d_{\lambda_t}};$$

4) 对 $j = 1, 2, \cdots, t-1$ 计算新的投影和分解系数:

$$\boldsymbol{\beta}_{j+1} = \boldsymbol{\beta}_j - \boldsymbol{\beta}_t \langle \Phi_{\lambda_t}, \boldsymbol{\beta}_j \rangle,$$
$$c_{j+1} = c_j - \overline{\langle \Phi_{\lambda_t}, \boldsymbol{\beta}_j \rangle} c_t;$$

5) 重复 2) ~ 4) 直到对于给定的小的正数 δ, 满足 $\|\boldsymbol{R}_t\|^2 \le \delta$ 时, 迭代停止, 并得到系数估计向量 $\boldsymbol{c}$, 且 $|\boldsymbol{c}| = M > m$;

6) 归一化 $\boldsymbol{c}$ 并向后剔除多余 $M - m$ 元素.

针对 OMP 中原子选择机制的缺点并引入简单的“回溯”思想,提出 IBOOMP 算法。模拟实验证明: IBOOMP 算法在相对误差一定的条件下较大程度地减少了重建对测量次数的要求,但是它与 MP 及 OMP 具有相同的最优原子选择机制, 每次迭代只选择一个原子, 为进一步提高重建速度及重建精度, 一次迭代选择多个原子的思想被采用到贪婪算法中.

> 方红,章权兵,韦穗.改进的后退型最优正交匹配追踪图像重建方法[J].华南理工大学学报(自然科学版),2008(08):23-27.

正则正交匹配追踪算法 ROMP

算法 4. ROMP 算法

1) 初始化残差 $\boldsymbol{r}_0 = \boldsymbol{y}$, 指标集合 $I = \emptyset$, 迭代 $t = 1$, 重复下面步骤 m 次或者直到 $|I| \geq 2m$;
2) 计算 $u = \langle \boldsymbol{r}_{t-1}, \Phi_j \rangle$;
3) 原子选择准则: m 个最大的非零坐标构成的集合或者它的所有非零坐标, 选其中较小的坐标设为 J;
4) 正则化: 在所有具有可比较坐标的子集 $J_0 \subset J$ 中: $|u(i)| \leq 2|u(j)|$, 对所有的 $i, j \in J_0$, 选择具有最大能量 $\|u|_{J_0}\|_{l_2}$ 的集合 J_0;
5) 更新: 增加 J_0 到指标集: $I \leftarrow I \cup J_0$, 更新残余:

$$\boldsymbol{x} = \arg \min_{\boldsymbol{z} \in \mathbf{R}^I} \|\boldsymbol{y} - \Phi \boldsymbol{z}\|_{l_2}, \boldsymbol{r} = \boldsymbol{y} - \Phi \boldsymbol{x}$$

MP 与 OMP 受矩阵列向量间相关性影响较大，而 ROMP 算法可以通过选择替代信号的最大元素并采用正则化的方式来确保没有太多的错误元素被选择，在每次迭代的过程中可以选多几个原子，ROMP 算法比 OMP 算法更快速且重构结果更加均衡稳定。

Matlab 实现

> D. Needell and R. Vershynin, "Signal Recovery From Incomplete and Inaccurate Measurements Via Regularized Orthogonal Matching Pursuit," in IEEE Journal of Selected Topics in Signal Processing , vol. 4, no. 2, pp. 310-316, April 2010, doi: 10.1109/JSTSP.2010.2042412.

子空间追踪 SP

算法 5. SP 算法

1) 初始化: $\widehat{T} = \{$相应于 $\Phi^{\mathrm{T}}\boldsymbol{y}$ 的绝对值最大的 m 个指标 $\}$
$\boldsymbol{y}_r = \mathrm{resid}(\boldsymbol{y}, \Phi_{\widehat{T}})$;
2) 迭代: 如果 $\boldsymbol{y}_r = 0$, 迭代停止; 否则继续;
3) $T' = \widehat{T} \cup \{$相应于 $\Phi^{\mathrm{T}}\boldsymbol{y}_r$ 的绝对值最大的 m 个指标$\}$
$\boldsymbol{x}'_p = \Phi^{\dagger}_{T'}\boldsymbol{y}$, $\widetilde{T} = \{$相应于 $\boldsymbol{x}'_p$ 最大元的 m 个指标$\}$,
$\widetilde{\boldsymbol{y}}_r = \mathrm{resid}(\boldsymbol{y}, \Phi_{\widetilde{T}})$;
4) 更新: 若 $\|\widetilde{\boldsymbol{y}}_r\|_{l_2} > \|\boldsymbol{y}_r\|_{l_2}$, 迭代停止; 否则, 设
$\widehat{T} = \widetilde{T}$ 以及 $\boldsymbol{y}_r = \widetilde{\boldsymbol{y}}_r$, 继续迭代;
输出: 满足 $\widehat{\boldsymbol{x}}_{\{1,2,\cdots,N\}-\widehat{T}} = 0$ 和 $\boldsymbol{x}_{\widehat{T}} = \Phi^{\dagger}_{\widehat{T}}\boldsymbol{y}$ 的估计信号 $\widehat{\boldsymbol{x}}$.

ROMP 和 SP 具有相同的原子选择机制, 每次迭代都以 $\phi^{T}\boldsymbol{y_r}$(Tyr 表示当前冗余) 中前 m 个绝对值最大的非零坐标所对应的原子作为候选; 其中,ROMP 利用正则化的方法找出候选原子集合中能量最

大的原子集合J_0, 在所有支撑集为J_0 的稀疏信号中利用最小二乘法找出使得$||y-\Phi x||_{l_2}$最小的解 x 作为当前估计, 并以$y-\Phi x$作为残差更新公式, 没有”回溯” 的思想, 每次迭代都在一个“可靠” 候选集中选择出满足正则条件的原子添加到上次迭代中, 并且这些原子一直保持到迭代停止. SP 与 ROMP 在残差更新方式上则不同, 它以相邻两次迭代冗余的差作为迭代停止条件, 每次迭代中在当次迭代支撑和前次迭代支撑集合的并集(T')上利用最小二乘求出当前估计$x=y$, 可见 SP 采用了“回溯”思想, 与 ROMP 相比更好地保证了重建的最优性.

Matlab 实现

> Dai W, Milenkovic O. Subspace pursuit for compressive sensing signal reconstruction[J]. IEEE transactions on Information Theory, 2009, 55(5): 2230-2249.

逐步正交匹配追踪 StOMP

算法 6. StOMP 算法

1) 初始解 $\boldsymbol{x}=0$, 初始残差 $\boldsymbol{r}_0=\boldsymbol{y}$, 计数器 $s=1$;

2) 第 s 阶段, 计算 $C_s=\Phi^{\mathrm{T}}\boldsymbol{r}_{s-1}=\langle\Phi,\boldsymbol{r}_{s-1}\rangle$;

3) 设立软阈值 $h=t_s\sigma_s$, 获得该阈值下的原子集合 $Js=\{j:|C_s(j)|>t_s\sigma_s\}$, 其中, 规范噪音水平为 $\sigma_s=\|r_s\|_{l_2}/\sqrt{M}$, 阈值参数 $2\le t_s\le 3$;

4) $I_s=I_{s-1}\cup J_s$, 计算支撑 I_s 上的逼近系数向量:

$$(\boldsymbol{x}_s)_{I_s}=(\Phi_{I_s}^{\mathrm{T}}\Phi_{I_s})^{-1}\Phi_{I_s}^{\mathrm{T}}\boldsymbol{y};$$

5) 更新残差为: $\boldsymbol{r}_s=\boldsymbol{y}-\Phi\boldsymbol{x}_s$;

6) 检查终止条件 (例如 $s=10$), 若未到停止时间, 则假设 $s=s+1$, 转到第 2) 步, 若到达停止时间, 设 $\hat{\boldsymbol{x}}_s=\boldsymbol{x}_s$, 为算法的最终输出.

StOMP 算法也是基于 OMP 进行改进的，每次迭代可以选择多个原子。但它的输入参数没有信号稀疏度 K，因此跟 ROMP 和 CoSaMP 有自己的优势。

Matlab 实现

> Donoho D L, Tsaig Y, Drori I, et al. Sparse solution of underdetermined linear equations by stagewise orthogonal matching pursuit, submitted to[C]//IEEE Trans. on Information theory. 2006.

压缩感知匹配追踪 CoSaMP

算法 7. CoSaMP 算法

输入: 采样矩阵算子 Φ, 噪音测量向量 $\boldsymbol{y}$, 稀疏度 m

输出: 待估信号 $\boldsymbol{x}$ 一个 m-稀疏逼近 $\boldsymbol{a}$

1) 初始解 $\boldsymbol{a}^0 = 0$, 初始残差 $\boldsymbol{r}_0 = \boldsymbol{y}$, 计数器 $s = 1$;

2) 计算信号代理: $\boldsymbol{c} = \Phi^{\mathrm{T}} \boldsymbol{r}_{s-1}$;

3) 鉴定大的成分, 合并支撑集: $\Omega = \mathrm{supp}(\boldsymbol{c}_{2m})$,

$T = \Omega \cup \mathrm{supp}(\boldsymbol{a}^{s-1})$;

4) 通过最小二乘法的信号估计: $\boldsymbol{b}|_T = \Phi_T^{\dagger} \boldsymbol{y}$,

$\boldsymbol{b}|_{T^c} = 0$ 从而获得新的逼近 $\boldsymbol{a}^s = \boldsymbol{b}_m$;

5) 更新残差: $\boldsymbol{r}_s = \boldsymbol{y} - \Phi \boldsymbol{a}^s$;

6) 若满足停止标准, 则输出 $\boldsymbol{a} = \boldsymbol{a}^s$, 若不满足, 则可以令

$s = s + 1$, 回到第 2) 步.

CoSaMP 也是对 OMP 的一种改进，每次迭代选择多个原子，除了原子的选择标准之外，它有一点不同于 ROMP：
ROMP 每次迭代已经选择的原子会一直保留，而 CoSaMP 每次迭代选择的原子在下次迭代中可能会被抛弃。

> Needell D, Tropp J A. CoSaMP: Iterative signal recovery from incomplete and inaccurate samples[J]. Applied and computational harmonic analysis, 2009, 26(3): 301-321.

StOMP 与 CoSaMP 在本质上都采用了“代理”的思想, 即如果测量矩阵满足限制等容常数 $\delta_m << 1$, 则向量$\boldsymbol{v} = \Phi^T \Phi \boldsymbol{x} = \Phi^T \boldsymbol{y}$ 可以看做待估信号 x 的一个“代理”, 其合理性在于: 根据 RIC 的定义, 当 $\delta_m << 1$ 时, v 中每 m 个元素构成的集合所包含的信息近似于 x 中 m 个元素所构成的集合,那么从 v 中前 m 个最大元所对应的支撑也应该对应于这些元素在 x 中的支撑.
StOMP 与 CoSaMP 的区别在于最优匹配原子的选择机制不同, StOMP 通过设立软阈值 h = tsδs

直接判断入选的原子集合, 利用最小二乘求出待估信号在所得支撑上的系数, 其缺点在于阈值参数选择较难; CoSaMP 则在每次迭代中计算代理信号(即 $\phi^T r_{s-1}$) 中前 2m 个最大元所对应的支撑与前次逼近解的支撑并作为当次候选支撑, 再利用最小二乘求出该支撑上的系数估计. 从算法 6 的第 4) 步和算法 7 的第 3) 步可以看出两者都采用了“回溯”思想.

分段弱正交匹配追踪 SWOMP

SWOMP 可以说是 StOMP 的一种改进算法，它们的唯一不同是选择原子时的门限设置，这可以降低对测量矩阵的要求。我们称这里的原子选择方式为“弱选择”。

Matlab 实现

> Blumensath T, Davies M E. Stagewise weak gradient pursuits[J]. IEEE Transactions on Signal Processing, 2009, 57(11): 4333-4346.

广义正交匹配追踪 GOMP

GOMP 可视为 OMP 算法的推广，OMP 每次只选择与残差相关最大的一个，而 GOMP 则是简单地选择最大的 S 个。

Matlab 实现

> Wang J, Kwon S, Shim B. Generalized orthogonal matching pursuit[J]. IEEE Transactions on signal processing, 2012, 60(12): 6202-6216.

稀疏度自适应匹配追踪 SAMP

SAMP 提出了一种不需要输入稀疏度 K 的算法，通过设置步长（很大程度影响速度、小程度影响精度）和合适的停止条件（很大程度影响精度）来进行稀疏度的估计和支撑集的填充。

Matlab 实现

> Do T T, Gan L, Nguyen N, et al. Sparsity adaptive matching pursuit algorithm for practical compressed sensing[C]//2008 42nd Asilomar conference on signals, systems and computers. IEEE, 2008: 581-587.

凸松弛类算法

稀疏表征模型在将 l_0 范数替换为 l_1 范数后约束优化问题形式：

$$\min_x ||x||_1 \quad s.t. ||y - \Phi x||_2 < \sigma$$

非约束优化问题形式：

$$\min_x ||y - \Phi x||_2 + \lambda ||x||_1$$

Candes，Tao 和 Donoho 等人已证明，当测量矩阵 Φ 满足约束等距性质(Restricted IsometryProperty，RIP) 时，组合优化问题(l_0 优化问题) 可转化为数值上容易处理 l_1 约束的凸优化问题

> 李珅, 马彩文, 李艳, 等. 压缩感知重构算法综述[D]. , 2013.

基追踪算法

l_1 范数约束优化问题等价于下面的线性规划问题：

$$\min c^{\mathrm{T}}x \quad Ax=b,\ x\geq 0$$

Matlab 实现

> Chen S S, Donoho D L, Saunders M A. Atomic decomposition by basis pursuit[J]. SIAM review, 2001, 43(1): 129-159.

迭代加权最小二乘法 IRLS

核心思想是通过将 L0-norm 松弛化（relaxation），把高度不连续的问题松弛化为连续的甚至是光滑逼近（smooth approximation）。

> Gorodnitsky I F, Rao B D. Sparse signal reconstruction from limited data using FOCUSS: A re-weighted minimum norm algorithm[J]. IEEE Transactions on signal processing, 1997, 45(3): 600-616.
>
> 详解

迭代硬/软阈值算法 IHT/IST、二步迭代软阈值算法 TwIST

其核心思想均为 Majorization-Minimization 思想，使用一个易优化的函数在一定条件下无限逼近原函数的最优解。

IHT 使用硬阈值函数，IST 使用软阈值函数，TwIST 为改进的 IST，收敛速度更快

IST 的 Matlab 实现

TwIST 的 Matlab 实现

> Blumensath T, Davies M E. Iterative hard thresholding for compressed sensing[J]. Applied and computational harmonic analysis, 2009, 27(3): 265-274.
>
> Bredies K, Lorenz D. Iterative soft-thresholding converges linearly[R]. Zentrum für Technomathematik, 2007.
>
> Bioucas-Dias J M, Figueiredo M A T. A new TwIST: Two-step iterative shrinkage/thresholding algorithms for image restoration[J]. IEEE Transactions on Image

processing, 2007, 16(12): 2992-3004.

其它迭代阈值算法：

Yuan X T , Li P , Zhang T . Gradient hard thresholding pursuit[J]. Journal of Machine Learning Research, 2018, 18:1-43.

杨海蓉, 方红, 张成,等. 基于回溯的迭代硬阈值算法[J]. 自动化学报, 2011(03):276-282.(Yang H R , Fang H , Zhang C , et al. Iterative Hard Thresholding Algorithm Based on Backtracking[J]. Acta Automatica Sinica, 2011, 37(3):276-282.)

Bahmani, Sohail, Raj, et al. Greedy Sparsity-Constrained Optimization.[J]. Journal of Machine Learning Research, 2013.

杨立波, 蒋铁钢, 徐志强. 基于混合梯度的硬阈值追踪算法[J]. 计算机应用, 2020, v.40;No.355(03):298-302.

其它算法

稀疏贝叶斯算法 MSBL

根据信号稀疏先验特性，将非线性优化问题转化为带有稀疏约束的线性回归问题的稀疏贝叶斯算法(MSBL)，因其具有更高的重构精度被越来越多地用于压缩感知重构中。

Wipf D P, Rao B D. An empirical Bayesian strategy for solving the simultaneous sparse approximation problem[J]. IEEE Transactions on Signal Processing, 2007, 55(7): 3704-3716.